NAME: TANKISO MASOEBE

INTRO TO DATABASES WEEK 12

1.Normalization Case Study Report

This case study analyzes a single unnormalized table representing an online retail dataset:

Sales Table
(SaleID, CustomerName, CustomerEmail, ProductID, ProductName, Category, SupplierName, SupplierContact, Quantity, UnitPrice)

Functional Dependencies

The following functional dependencies (FDs) can be identified:

- SaleID → CustomerName, CustomerEmail
- ProductID → ProductName, Category, SupplierName
- SupplierName → SupplierContact
- ProductID → UnitPrice
- SaleID, ProductID → Quantity

Current Normal Form Analysis

The table is in **First Normal Form (1NF)** because all attributes contain atomic values (no repeating groups or arrays). However, it violates higher normal forms:

Violation of Second Normal Form (2NF)

The primary key is composite: (SaleID, ProductID).
Partial dependencies exist:

- SaleID → CustomerName, CustomerEmail
- ProductID → ProductName, Category, SupplierName, UnitPrice

These attributes depend only on part of the composite key, not the whole key, violating 2NF.

Violation of Third Normal Form (3NF)

There is a transitive dependency:

- SupplierName → SupplierContact
- ProductID → SupplierName

Thus, ProductID indirectly determines SupplierContact, violating 3NF.

Violation of Boyce-Codd Normal Form (BCNF)

BCNF requires that every determinant must be a candidate key.
However:

- SupplierName → SupplierContact
  SupplierName is not a candidate key, so BCNF is violated.

Step-by-Step Decomposition to BCNF

Step 1: Remove Partial Dependencies (2NF)

Decompose into:

1. **Sales** (SaleID, CustomerName, CustomerEmail)
2. **Products** (ProductID, ProductName, Category, SupplierName, UnitPrice)
3. **SalesDetails** (SaleID, ProductID, Quantity)

Step 2: Remove Transitive Dependencies (3NF)

From Products:

- SupplierName → SupplierContact

Decompose into:

4. **Suppliers** (SupplierName, SupplierContact)
5. **Products** (ProductID, ProductName, Category, SupplierName, UnitPrice)

tep 3: Ensure BCNF Compliance

Check all tables:

- **Sales**: SaleID is the key → satisfies BCNF

- **SalesDetails**: (SaleID, ProductID) is the key → satisfies BCNF
- **Products**: ProductID determines all attributes → satisfies BCNF
- **Suppliers**: SupplierName is the key → satisfies BCNF

Final BCNF Schema

- Sales (SaleID, CustomerName, CustomerEmail)
- SalesDetails (SaleID, ProductID, Quantity)
- Products (ProductID, ProductName, Category, SupplierName, UnitPrice)
- Suppliers (SupplierName, SupplierContact)

Conclusion

The original table violated 2NF, 3NF, and BCNF due to partial and transitive dependencies. Through systematic decomposition, redundancy and anomalies were eliminated, resulting in a BCNF-compliant schema that ensures data integrity and efficient database design.

Denormalization is the intentional process of introducing redundancy into a database by combining tables or duplicating data that would otherwise be separated in a fully normalized design. While normalization (up to 3NF or BCNF) minimizes redundancy and ensures strong data integrity, database administrators sometimes denormalize systems to improve performance—especially in high-read environments like data warehouses or reporting systems.

One major reason for denormalization is to **optimize complex queries**. In normalized databases, retrieving

2.**Debate Summary: Normalization vs. Performance (Denormalization)**

related data often requires multiple joins across tables. For large datasets, these joins can become computationally expensive and slow. By merging frequently accessed tables or storing precomputed values (e.g., totals, counts), denormalization reduces the need for joins, resulting in faster query execution. This is particularly useful in business intelligence systems where speed is critical for dashboards and analytics.

Another advantage is improved **read performance**. Applications that rely heavily on reading data (such as e-commerce platforms or analytics dashboards) benefit from fewer joins and simpler queries. Denormalized tables can store all relevant information in one place, allowing quicker access and reduced database load.

However, denormalization comes with important trade-offs. The most significant is **reduced data integrity**. Since data is duplicated, inconsistencies can occur if updates are not applied uniformly across all copies. For example, updating a customer's email in one table but not another leads to anomalies. This increases the complexity of maintaining accurate data.

Additionally, denormalized databases may suffer from **update anomalies** and increased storage requirements due to redundancy. Insertions, updates, and deletions become more error-prone and harder to manage compared to a normalized design.

In conclusion, denormalization is a strategic decision used to balance performance and integrity. While it sacrifices some consistency and increases maintenance complexity, it significantly enhances read efficiency, making it suitable for systems where fast data retrieval is more critical than strict normalization.